

CA-CFAR FPGA Accelerator Architecture (PYNQ-Z2) - Detailed Explanation

1. Overview

This project implements a hardware-accelerated 2D Cell Averaging CFAR (CA-CFAR) detector for SAR ship detection on a PYNQ-Z2 FPGA. SAR image pixels are streamed into the FPGA through AXI DMA. The FPGA performs real-time CFAR detection and returns a detection mask to the ARM processor.

2. Processing Flow

SAR Image -> AXI DMA -> AXI4-Stream Wrapper -> CFAR Core -> Detection Frame Buffer -> AXI DMA -> Python Visualization.

3. CFAR Geometry

GUARD=1 and TRAIN=2 create a 7x7 processing window. The center 3x3 region contains the Cell Under Test (CUT) and guard cells. The remaining 40 cells are training cells used to estimate local clutter power.

4. Raster Scan Input

Pixels arrive one per clock in raster order. Row and column counters track the location of every incoming pixel.

5. Line Buffers

Six line buffers store previous image rows. These buffers allow the FPGA to build a complete 7x7 neighborhood around every pixel while processing a continuous stream.

6. Sliding Window Generator

A 7x7 shift-register window moves across the image. Every clock cycle the window shifts left and a new column is inserted from the line buffers and current input pixel.

7. Training Cell Accumulation

The CFAR engine visits all 49 cells in the window. The 3x3 guard region is excluded and only the 40 training cells are summed to estimate background clutter.

8. Cell Under Test (CUT)

The center pixel of the 7x7 window is selected as the Cell Under Test. This pixel is compared against an adaptive threshold.

9. Threshold Calculation

Instead of performing division, the implementation uses integer arithmetic. The comparison is equivalent to $CUT > \alpha \times \text{average clutter}$. With $ALPHA=640$ and $ALPHA_FRAC=8$, the threshold factor is 2.5.

10. Detection Decision

If the CUT exceeds the adaptive threshold, a detection bit is asserted. Otherwise the output remains zero.

11. Interior Pixel Validation

Border pixels do not have a complete 7x7 neighborhood. Therefore detections are only considered valid for interior pixels.

12. Output Coordinates

For every valid detection, the module outputs the detection bit along with row and column coordinates.

13. AXI4-Stream Wrapper

The `cfar2d_axis` module wraps the detector for AXI DMA communication and frame-based processing.

14. Frame Buffer

Detection results are stored in a one-bit-per-pixel frame buffer. This frame buffer represents the binary detection mask.

15. FSM States

LOAD: Receive image pixels. DRAIN: Flush remaining pipeline data. SEND: Stream detection mask back through DMA. DONE: Return to LOAD state for the next frame.

16. DMA Operation

Python transfers image tiles to the FPGA through MM2S DMA. The FPGA processes the tile and sends the detection mask back through S2MM DMA.

17. SAR Ship Detection Use Case

Bright ship returns appear above the surrounding sea clutter. CFAR adapts the threshold locally, enabling detection under varying sea conditions.

18. FPGA Skills Demonstrated

Verilog RTL Design, AXI4-Stream Interfaces, DMA Integration, Sliding Window Architectures, Line Buffers, Fixed-Point Arithmetic, DSP Processing, Embedded Linux Integration, and Hardware/Software Co-Design.

19. Interview Summary

Implemented a streaming 2D CA-CFAR detector on PYNQ-Z2 using a 7x7 sliding window and six line buffers. The accelerator processes one pixel per clock, computes adaptive thresholds from 40 training cells, and performs FPGA-accelerated SAR ship detection using AXI DMA.